

只要提供适当的署名，谷歌特此授权仅出于新闻或
学术著作。

注意力就是你所需要的一切

Ashish Vaswani* Google Brain
avaswani@google.com

诺姆·沙泽尔* Google Brain
noam@google.com

Niki Parmar* Google Research
nikip@google.com

Jakob Uszkoreit* Google Research
usz@google.com

Llion Jones* 谷歌研究
llion@google.com

Aidan N. Gomez*[†] 多伦多大学
aidan@cs.toronto.edu

ukasz Kaiser* Google Brain
lukasz.kaiser@google.com

伊利亚·波洛苏欣*[‡]

illia.polosukhin@gmail.com

摘要

主流的序列转导模型基于复杂的循环或卷积神经网络，这些网络包含一个编码器和一个解码器。性能最好的模型还通过注意力机制将编码器和解码器连接起来。我们提出了一种新的简单网络架构，即Transformer，它完全基于注意力机制，彻底摒弃了循环和卷积。在两个机器翻译任务上的实验表明，这些模型在质量上更优，同时更易于并行化，且训练所需时间显著减少。我们的模型在WMT 2014英德翻译任务上达到了28.4 BLEU，比包括集成模型在内的现有最佳结果提高了超过2 BLEU。在WMT 2014英法翻译任务上，我们的模型在8个GPU上训练3.5天后，创下了41.8的单模型最佳BLEU分数，这仅为文献中最佳模型训练成本的一小部分。我们通过将Transformer成功应用于具有大量和有限训练数据的英语成分句法分析，证明了该模型能够很好地泛化到其他任务。

^{*}同等贡献。作者排序是随机的。Jakob提出用自注意力机制取代循环神经网络，并开始评估这一想法。Ashish与Illia设计并实现了首个Transformer模型，并关键性地参与了这项工作的方方面面。Noam提出了缩放点积注意力、多头注意力以及无参数位置表示，成为参与几乎所有细节的另一人。Niki在我们最初的代码库和tensor2tensor中设计、实现、调整并评估了无数的模型变体。Llion也尝试了新型模型变体，负责我们最初的代码库、高效推理和可视化工作。Lukasz和Aidan花费了无数个漫长的日子设计和实现tensor2tensor的各个部分，替换了我们早期的代码库，极大地提升了结果并大幅加速了我们的研究。

[†] 在 Google Brain 工作期间完成的工作。

[‡] 在谷歌研究院工作期间完成的工作。

1 引言

循环神经网络，特别是长短期记忆[13]和门控循环[7]神经网络，已在序列建模和转换问题（如语言建模和机器翻译[35, 2, 5]）中被确立为最先进的方法。此后，大量研究持续推动着循环语言模型和编码器-解码器架构[38, 24, 15]的边界。

循环模型通常沿输入和输出序列的符号位置进行计算分解。通过将位置与计算时间步对齐，它们生成一系列隐藏状态 h_t ，该状态是前一隐藏状态 h_{t-1} 和位置 t 的输入的函数。这种固有的顺序特性排除了训练样本内的并行化，这在序列长度较长时变得至关重要，因为内存限制了跨样本的批处理。最近的研究通过分解技巧 [21] 和条件计算 [32] 在计算效率方面取得了显著改进，同时在后者的情况下还提升了模型性能。然而，顺序计算的根​​本约束依然存在。

注意力机制已成为各种任务中引人注目的序列建模和转换模型不可或缺的一部分，允许在不考虑输入或输出序列中距离的情况下对依赖关系进行建模 [2, 19]。然而，除了少数情况 [27] 之外，此类注意力机制通常与循环网络结合使用。

在这项工作中，我们提出了Transformer，这是一种摒弃了循环结构，转而完全依赖注意力机制来建立输入与输出之间全局依赖关系的模型架构。Transformer允许实现显著的并行化，并且在八块P100 GPU上仅训练十二小时后，就能在翻译质量上达到新的最先进水平。

2 背景

减少顺序计算的目标也构成了扩展神经GPU [16]、ByteNet [18] 和 ConvS2S [9] 的基础，这些模型都使用卷积神经网络作为基本构建模块，为所有输入和输出位置并行计算隐藏表示。在这些模型中，将两个任意输入或输出位置的信号联系起来所需的计算量随位置距离增加而增长，ConvS2S 为线性增长，ByteNet 为对数增长。这使得学习远距离位置之间的依赖关系变得更加困难 [12]。在 Transformer 中，这一计算量被减少到常数次操作，尽管代价是由于对注意力加权位置进行平均而导致有效分辨率降低，我们通过第 3.2 节中描述的多头注意力机制来抵消这一影响。

自注意力机制，有时被称为内部注意力机制，是一种将单个序列的不同位置联系起来以计算序列表示的注意力机制。自注意力机制已成功应用于多种任务，包括阅读理解、生成式摘要、文本蕴含以及学习任务无关的句子表示 [4, 27, 28, 22]。

端到端记忆网络基于循环注意力机制而非序列对齐的循环，并已被证明在简单语言问答和语言建模任务中表现良好 [34]。

据我们所知，Transformer 是第一个完全依赖自注意力机制来计算输入和输出表示的转换模型，而无需使用序列对齐的循环神经网络或卷积。在接下来的章节中，我们将描述 Transformer，阐述自注意力机制的动机，并讨论其相对于 [17, 18] 和 [9] 等模型的优势。

3 模型架构

大多数具有竞争力的神经序列转换模型都采用编码器-解码器结构 [5, 2, 35]。在这里，编码器将符号表示的输入序列 $(x_1, \dots, x_n)$ 映射为连续表示序列 $z = (z_1, \dots, z_n)$ 。给定 z ，解码器随后逐个元素生成符号输出序列 $(y_1, \dots, y_m)$ 。在每一步中，该模型都是自回归的[10]，在生成下一个符号时，将先前生成的符号作为额外输入。

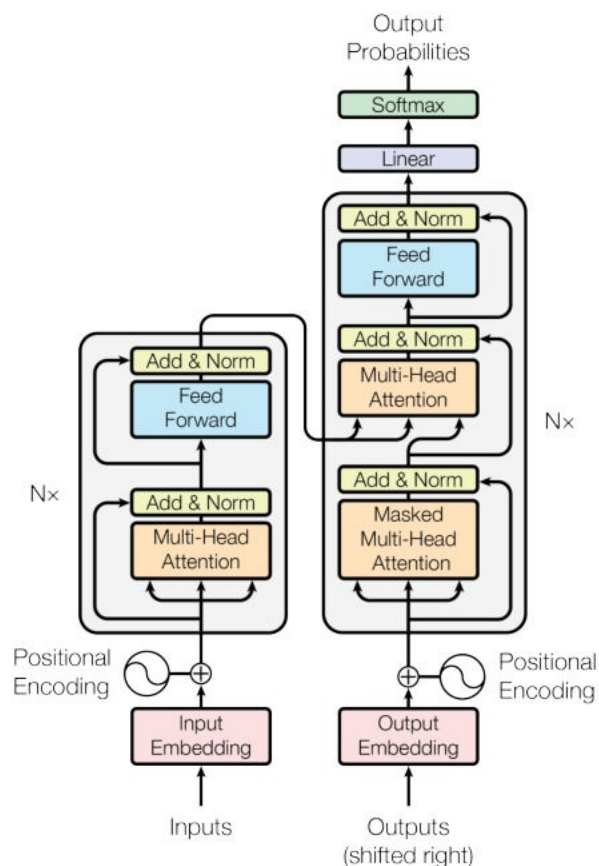


图1: Transformer - 模型架构。

Transformer 遵循这种整体架构，在编码器和解码器中均使用堆叠的自注意力机制和逐点全连接层，分别如图 1 的左半部分和右半部分所示。

3.1 编码器和解码器堆栈

编码器： 编码器由 $N = 6$ 个相同的层堆叠而成。每一层包含两个子层。第一个是多头自注意力机制，第二个是简单的逐位置全连接前馈网络。我们在两个子层周围都采用了残差连接 [1]，随后进行层归一化 [1]。也就是说，每个子层的输出为 $\text{LayerNorm}(x + \text{Sublayer}(x))$ ，其中 $\text{Sublayer}(x)$ 是子层本身实现的函数。为了便于这些残差连接，模型中的所有子层以及嵌入层都产生维度为 $d_{\text{model}} = 512$ 的输出。

解码器： 解码器同样由 $N = 6$ 个相同的层堆叠而成。除了每个编码器层中的两个子层外，解码器还插入了第三个子层，该子层对编码器堆栈的输出执行多头注意力机制。与编码器类似，我们在每个子层周围采用残差连接，随后进行层归一化。我们还修改变码器堆栈中的自注意力子层，以防止位置关注后续位置。这种掩码机制，结合输出嵌入偏移一个位置的事实，确保了位置 i 的预测只能依赖于小于 i 的已知位置的输出。

3.2 注意力

注意力函数可以描述为将一个查询和一个键值对集合映射到一个输出，其中查询、键、值和输出都是向量。输出被计算为加权和

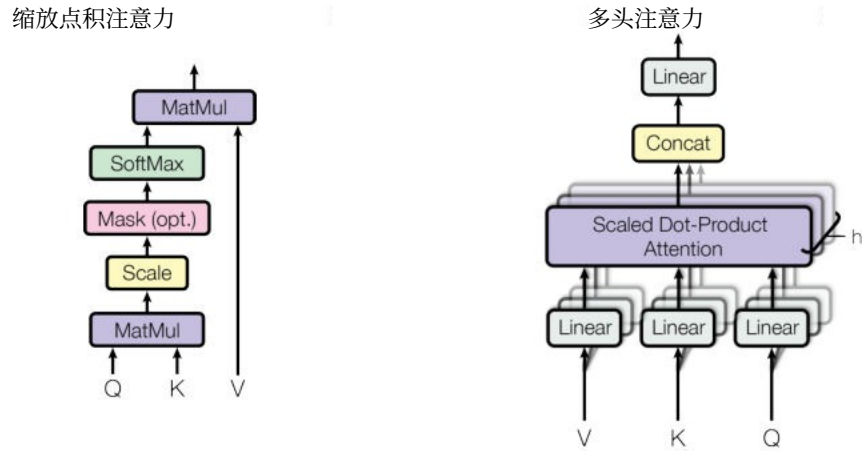


图2: (左) 缩放点积注意力机制。(右) 多头注意力机制由多个并行运行的注意力层组成。

这些值的，其中分配给每个值的权重是由查询与相应键的兼容性函数计算得出的。

3.2.1 缩放点积注意力

我们将这种特殊的注意力机制称为“缩放点积注意力”（图2）。输入由维度为 d_k d_v 的查询和键组成。我们计算查询与所有键的点积，将每个点积除以 $\sqrt{d_k}$ ，并应用 softmax 函数以获得作用于值上的权重。

在实践中，我们同时对一组查询计算注意力函数，将它们打包成一个矩阵 Q 。键和值也被打包成矩阵 K 和 V 。我们计算输出矩阵如下：

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V \quad (1)$$

最常用的两种注意力函数是加性注意力 [2] 和点积（乘性）注意力。点积注意力与我们的算法完全相同，除了缩放因子 $1/\sqrt{d_k}$ 之外。加性注意力使用带有单个隐藏层的前馈网络来计算兼容性函数。虽然两者在理论复杂度上相似，但在实际应用中，点积注意力速度更快且更节省空间，因为它可以使用高度优化的矩阵乘法代码来实现。

虽然对于较小的 d_k 值，两种机制表现相似，但对于较大的 d_k 值，加法注意力优于未缩放的点积注意力[3]。我们怀疑对于较大的 d_k 值，点积的绝对值会变得很大，从而将 softmax 函数推向梯度极小的区域⁴。为了抵消这种影响，我们通过 $1/\sqrt{d_k}$ 对点积进行缩放。

3.2.2 多头注意力

与其使用 d_{model} 维度的键、值和查询执行单一的注意力函数，我们发现将查询、键和值分别通过 h 次不同的、可学习的线性投影，线性投影到 d_k 、 d_k 和 d_v 维度是有益的。在这些投影后的查询、键和值的每一个版本上，我们随后并行执行注意力函数，产生 d_v 维度的

⁴为了说明为什么点积会变大，假设 q 和 k 的分量是均值为 0、方差为 1 的独立随机变量。那么它们的点积， $q \cdot k = \sum_{i=1}^{d_k} q_i k_i$ ，均值为 0，方差为 d_k 。

输出值。这些值被拼接并再次投影，从而得到最终值，如图2所示。

多头注意力机制允许模型同时关注来自不同位置、不同表示子空间的信息。使用单个注意力头时，平均操作会抑制这种能力。

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) W^O$$

$$\text{where head}_i = \text{Attention}(Q W_i^Q, K W_i^K, V W_i^V)$$

其中投影为参数矩阵 $W_i^Q \in \mathbb{R}^{d_{\text{model}} \times d_k}$, $W_i^K \in \mathbb{R}^{d_{\text{model}} \times d_k}$, $W_i^V \in \mathbb{R}^{d_{\text{model}} \times d_v}$ 和 $W^O \in \mathbb{R}^{d_v \times d_{\text{model}}}$ 。

在这项工作中，我们采用 $h = 8$ 个并行注意力层，或称多头。对于其中的每一个，我们使用 $d_k = d_v = d_{\text{model}}/h = 64$ 。由于每个头的维度降低，总计算成本与具有完整维度的单头注意力相似。

3.2.3 注意力机制在我们模型中的应用

Transformer 在三种不同方式下使用多头注意力机制：

- 在“编码器-解码器注意力”层中，查询来自前一个解码器层，而记忆键和值来自编码器的输出。这使得解码器中的每个位置都能关注输入序列中的所有位置。这模拟了序列到序列模型中典型的编码器-解码器注意力机制，例如 [38, 2, 9]。
- 编码器包含自注意力层。在自注意力层中，所有的键、值和查询都来自同一处，在本例中，即来自编码器中前一层的输出。编码器中的每个位置都可以关注编码器前一层的所有位置。
- 同样地，解码器中的自注意力层允许解码器中的每个位置关注解码器中直到且包括该位置的所有位置。我们需要防止解码器中的左向信息流，以保持自回归特性。我们在缩放点积注意力内部实现这一点，通过将softmax输入中对应于非法连接的所有值遮蔽（设置为 $-\infty$ ）。参见图2。

3.3 逐位置前馈网络

除了注意力子层外，我们编码器和解码器中的每一层都包含一个全连接前馈网络，该网络独立且相同地应用于每个位置。它由两个线性变换组成，中间夹着一个ReLU激活函数。

$$\text{FFN}(x) = \max(0, xW_1 + b_1)W_2 + b_2 \quad (2)$$

虽然线性变换在不同位置是相同的，但它们在层与层之间使用不同的参数。另一种描述方式是将其视为两个卷积核大小为1的卷积。输入和输出的维度是 $d_{\text{model}} = 512$ ，而中间层的维度是 $d_{\text{ff}} = 2048$ 。

3.4 嵌入与Softmax

与其他序列转换模型类似，我们使用学习到的嵌入将输入标记和输出标记转换为维度为 d_{model} 的向量。我们还使用常规的学习线性变换和 softmax 函数将解码器输出转换为预测的下一个标记概率。在 our model, we share the same weight matrix between the two embedding layers and the pre-softmax linear transform, 类似于 [30]。在嵌入层中，我们将这些权重乘以 $\sqrt{d_{\text{model}}}$ 。

表 1: 不同层类型的最大路径长度、每层复杂度以及最小顺序操作数。n 是序列长度, d 是表示维度, k 是卷积的卷积核大小, r 是受限自注意力机制中的邻域大小。

图层类型	每层复杂度	连续的操作	最大路径长度
自注意力	$O(n^2 d)$	$O(1)$	$O(1)$
循环的	$O(n^2 d^2)$	$O(n)$	$O(n)$
卷积的	$O(k^n d^2)$	$O(1)$	$O(\log_k(n))$
自注意力 (受限)	$O(r^n d)$	$O(1)$	$O(n/r)$

3.5 位置编码

由于我们的模型不包含循环和卷积, 为了让模型能够利用序列的顺序, 我们必须注入一些关于序列中标记的相对或绝对位置的信息。为此, 我们在编码器和解码器堆栈底部的输入嵌入中添加“位置编码”。位置编码与嵌入具有相同的维度 d_{model} , 因此两者可以相加。位置编码有许多选择, 包括可学习的和固定的[9]。

在这项工作中, 我们使用了不同频率的正弦和余弦函数:

$$PE_{(pos, 2i)} = \sin(pos/10000^{2i/d_{\text{model}}})$$

$$PE_{(pos, 2i+1)} = \cos(pos/10000^{2i/d_{\text{model}}})$$

其中 pos 是位置, i 是维度。也就是说, 位置编码的每个维度都对应一个正弦波。波长形成一个从 2π 到 $10000 \cdot 2\pi$ 的几何级数。我们选择这个函数是因为我们假设它能让模型轻松地学习按相对位置进行注意力机制, 因为对于任何固定的偏移量 k, PE_{pos+k} 都可以表示为 PE_{pos} 的线性函数。

我们还尝试使用学习到的位置嵌入 [9] 代替, 并发现这两个版本产生了几乎相同的结果 (见表 3 行 (E))。我们选择了正弦版本, 因为它可能允许模型外推到比训练期间遇到的更长的序列长度。

4 为什么使用自注意力机制

在本节中, 我们将自注意力层的各个方面与通常用于将一个变长符号表示序列 $(x_1, \dots, x_n)$ 映射到另一个等长序列 $(z_1, \dots, z_n)$ 的循环层和卷积层进行比较, 其中 $x_i, z_i \in \mathbb{R}^d$, 例如典型序列转换编码器或解码器中的隐藏层。为了说明使用自注意力的动机, 我们考虑三个设计要求。

一个是每层的总计算复杂度。另一个是可以并行化的计算量, 以所需的最少顺序操作数来衡量。

第三是网络中长程依赖之间的路径长度。学习长程依赖是许多序列转换任务中的一个关键挑战。影响学习此类依赖能力的一个关键因素是前向和后向信号在网络中必须遍历的路径长度。输入和输出序列中任意位置组合之间的这些路径越短, 学习长程依赖就越容易 [12]。因此, 我们还比较了由不同层类型组成的网络中任意两个输入和输出位置之间的最大路径长度。

如表1所示, 自注意力层通过固定数量的顺序执行操作连接所有位置, 而循环层则需要 $O(n)$ 个顺序操作。在计算复杂度方面, 当序列长度较短时, 自注意力层比循环层更快。

长度 n 小于表示维度 d ，这在机器翻译中最先进模型所使用的句子表示中最为常见，例如word-piece [38]和byte-pair [31]表示。为了提高涉及极长序列任务的计算性能，自注意力机制可以被限制为仅考虑输入序列中以相应输出位置为中心、大小为 r 的邻域。这将把最大路径长度增加到 $O(n/r)$ 。我们计划在未来的工作中进一步研究这种方法。

单个卷积层如果卷积核宽度 $k < n$ ，则无法连接所有输入和输出位置的配对。若要实现这一点，在连续卷积核的情况下需要堆叠 $O(n/k)$ 个卷积层，或者在空洞卷积的情况下需要 $O(\log_k(n))$ 个卷积层 [18]，这会增加网络中任意两个位置之间最长路径的长度。卷积层通常比循环层更昂贵，其成本倍数为 k 。然而，可分离卷积 [6] 显著降低了复杂度，降至 $O(k \cdot n \cdot d + n \cdot d^2)$ 。即便在 $k = n$ 的情况下，可分离卷积的复杂度也等于自注意力层与逐点前馈层的组合，这正是我们模型所采用的方法。

作为附加好处，自注意力机制可以产生更具可解释性的模型。我们检查了模型的注意力分布，并在附录中展示和讨论了相关示例。不仅单个注意力头能清晰地学会执行不同的任务，许多注意力头似乎还表现出与句子语法和语义结构相关的行为。

5 训练

本节描述了我们模型的训练方案。

5.1 训练数据与批处理

我们在标准的WMT 2014英德数据集上进行了训练，该数据集包含约450万个句子对。句子使用字节对编码[3]进行编码，其共享的源-目标词汇表包含约37000个标记。对于英法翻译，我们使用了规模大得多的WMT 2014英法数据集，该数据集包含3600万个句子，并将标记拆分为一个32000词的子词词汇表[38]。句子对按近似序列长度进行批处理。每个训练批次包含一组句子对，其中包含约25000个源标记和25000个目标标记。

5.2 硬件与进度

我们在配备8块NVIDIA P100 GPU的单台机器上训练了我们的模型。对于使用本文所述超参数的基础模型，每个训练步骤耗时约0.4秒。我们总共训练了基础模型100,000个步骤，即12小时。对于大模型（见表3底行描述），每步耗时为1.0秒。大模型训练了300,000个步骤（3.5天）。

5.3 优化器

我们使用了Adam优化器[20]，其中 $\beta_1 = 0.9$ ， $\beta_2 = 0.98$ ， $\epsilon = 10^{-9}$ 。在训练过程中，我们根据以下公式调整学习率：

$$lr_{rate} = d_{\text{model}}^{-0.5} \cdot \min(step_num^{-0.5}, step_num \cdot warmup_steps^{-1.5}) \quad (3)$$

这对应于在前 $warmup_steps$ 个训练步中线性增加学习率，此后按步数的平方根倒数比例降低学习率。我们使用了 $warmup_steps = 4000$ 。

5.4 正则化

我们在训练过程中采用了三种类型的正则化：

表2: Transformer在英语-德语和英语-法语newstest2014测试中, 以极低的训练成本实现了比以往最先进模型更好的BLEU分数。

模型	BLEU		训练成本 (FLOPs)	
	英语-德语	英语-法语	英语-德语	英语-法语
ByteNet [18]	23.75			
Deep-Att + PosUnk [39]		39.2		$1.0 \cdot 10^{20}$
GNMT + RL [38]	24.6	39.92	$2.3 \cdot 10^{19}$	$1.4 \cdot 10^{20}$
ConvS2S [9]	25.16	40.46	$9.6 \cdot 10^{18}$	$1.5 \cdot 10^{20}$
MoE [32]	26.03	40.56	$2.0 \cdot 10^{19}$	$1.2 \cdot 10^{20}$
Deep-Att + PosUnk 集成 [39]		40.4		$8.0 \cdot 10^{20}$
GNMT + RL 集成 [38]	26.30	41.16	$1.8 \cdot 10^{20}$	$1.1 \cdot 10^{21}$
ConvS2S 集成 [9]	26.36	41.29	$7.7 \cdot 10^{19}$	$1.2 \cdot 10^{21}$
Transformer (基础模型)	27.3	38.1	$3.3 \cdot 10^{18}$	
Transformer (大)	28.4	41.8	$2.3 \cdot 10^{18}$	

9

残差丢弃 我们在每个子层的输出上应用丢弃 [33], 然后再将其与子层输入相加并进行归一化。此外, 我们在编码器和解码器堆栈中, 对嵌入和位置编码的和应用丢弃。对于基础模型, 我们使用的丢弃率 $P_{\text{drop}} = 0.1$ 。

标签平滑 在训练过程中, 我们采用了值为 $\epsilon_{\text{ls}} = 0.1$ 的标签平滑 [36]。这会损害困惑度, 因为模型学会变得更加不确定, 但能提高准确率和 BLEU 分数。

6 结果

6.1 机器翻译

在WMT 2014英德翻译任务上, 大型Transformer模型(表2中的Transformer (big))比此前报告的最佳模型(包括集成模型)高出2.0个BLEU分值以上, 创下了28.4的全新SOTA BLEU分数。该模型的配置列于表3的最后一行。训练在8块P100 GPU上耗时3.5天。即使是我们的基础模型, 也超过了所有此前发表的模型和集成模型, 且训练成本仅为任何竞争模型的一小部分。

在WMT 2014英法翻译任务上, 我们的big模型实现了41.0的BLEU得分, 优于所有先前发布的单模型, 且训练成本不到先前最先进模型的1/4。用于英法翻译的Transformer (big)模型使用了dropout率 $P_{\text{drop}} = 0.1$, 而非0.3。

对于基础模型, 我们使用了一个通过对最后5个检查点取平均值得到的单一模型, 这些检查点是以10分钟为间隔保存的。对于大型模型, 我们对最后20个检查点取平均值。我们使用了束搜索, 束大小为4, 长度惩罚系数 $\alpha = 0.6$ [38]。这些超参数是在开发集上进行实验后选定的。我们将推理时的最大输出长度设置为输入长度 + 50, 但在可能的情况下会提前终止 [38]。

表2总结了我们的结果, 并将我们的翻译质量和训练成本与其他文献中的模型架构进行了比较。我们通过将训练时间、使用的GPU数量以及每个GPU的持续单精度浮点能力估值相乘, 来估算训练模型所使用的浮点运算次数⁵。

6.2 模型变体

为了评估Transformer不同组件的重要性, 我们以不同方式改变了基础模型, 测量了在英德翻译任务上的性能变化, 在

⁵我们分别对K80、K40、M40和P100使用了2.8、3.7、6.0和9.5 TFLOPS的数值。

表 3: Transformer 架构的变体。未列出的数值与基础模型相同。所有指标均基于英语到德语翻译开发集 newstest2013。列出的困惑度是基于我们的字节对编码的每词元困惑度，不应与每词困惑度进行比较。

	N	dmodel	d _{ff}	h	d _k	d _v	Pdrop	ϵ _{ls}	训练 步骤	PPL (开发)	BLEU (开发)	参数 ₆ ×10 ⁶
基础	6	512	2048	8	64	64	0.1	0.1	100K	4.92	25.8	65
(A)					1	512				5.29	24.9	
					4	128				5.00	25.5	
					16	32				4.91	25.8	
					32	16				5.01	25.4	
(B)					16					5.16	25.1	58
					32					5.01	25.4	60
(C)	2									6.11	23.7	36
	4									5.19	25.3	50
	8									4.88	25.5	80
					32	32				5.75	24.5	28
					128	128				4.66	26.0	168
					1024					5.12	25.4	53
(D)										4.75	26.2	90
							0.0			5.77	24.6	
							0.2			4.95	25.5	
							0.0			4.67	25.3	
(E)							0.2			5.47	25.7	
	位置嵌入代替正弦函数									4.92	25.7	
大	6	1024	4096	16				0.3	300K	4.33	26.4	213

开发集，newstest2013。我们使用了前一节所述的束搜索，但没有进行检查点平均。我们在表3中展示了这些结果。

在表3的(A)行中，我们改变了注意力头的数量以及注意力键和值的维度，同时保持计算量不变，如第3.2.2节所述。虽然单头注意力的BLEU得分比最佳设置低0.9，但当注意力头过多时，质量也会下降。

在表3的(B)行中，我们观察到减小注意力键的大小 d_k 会损害模型质量。这表明确定兼容性并不容易，并且使用比点积更复杂的兼容性函数可能会有益。我们进一步在(C)和(D)行中观察到，正如预期的那样，更大的模型效果更好，且dropout对于避免过拟合非常有帮助。在(E)行中，我们将正弦位置编码替换为学习的位置嵌入[9]，并观察到与基础模型几乎相同的结果。

6.3 英语成分句法分析

为了评估Transformer是否能泛化到其他任务，我们进行了英语成分句法分析实验。这项任务面临着特定的挑战：输出受到强结构约束，且明显长于输入。此外，RNN序列到序列模型在小数据量情况下无法达到最先进的结果[37]。

我们在宾州树库（Penn Treebank）[25]的《华尔街日报》（WSJ）部分上训练了一个4层Transformer， $d_{\text{model}} = 1024$ ，包含约4万条训练句子。我们还在半监督设置下对其进行了训练，使用了包含约1700万条句子的更大规模的高置信度语料库和BerkleyParser语料库[37]。对于仅WSJ的设置，我们使用了1.6万个词汇的词表；对于半监督设置，我们使用了3.2万个词汇的词表。

我们仅进行了少量的实验，在第22节开发集上选择丢弃率、注意力与残差（第5.4节）、学习率和束搜索大小，所有其他参数均保持与英语到德语基础翻译模型一致。在推理过程中，我们

表 4: Transformer 在英语成分句法分析上具有良好的泛化能力 (结果基于 WSJ 第 23 节)

解析器	训练	WSJ 23 F1
Vinyals & Kaiser 等人 (2014) [37]	WSJ only, 判别式	88.3
Petrov 等人 (2006) [29]	WSJ only, 判别式	90.4
Zhu 等人 (2013) [40]	WSJ only, 判别式	90.4
Dyer 等人 (2016) [8]	WSJ only, 判别式	91.7
Transformer (4层)	WSJ only, 判别式	91.3
Zhu 等人 (2013) [40]	半监督	91.3
Huang & Harper (2009) [14]	半监督	91.3
McClosky 等人 (2006) [26]	半监督	92.1
Vinyals & Kaiser 等人 (2014) [37]	半监督	92.1
Transformer (4层)	半监督	92.7
Luong 等人 (2015) [23]	多任务	93.0
Dyer 等人 (2016) [8]	生成式	93.

3

将最大输出长度增加到输入长度 + 300。我们在仅 WSJ 和半监督设置中均使用了 21 的束搜索大小和 $\alpha = 0.3$ 。

表4中的结果显示, 尽管缺乏针对特定任务的微调, 我们的模型表现依然出奇地好, 取得了比之前所有已报道模型更好的结果, 除了循环神经网络语法[8]之外。

与RNN序列到序列模型[37]相比, Transformer即使仅在包含4万个句子的WSJ训练集上进行训练, 其表现也优于BerkeleyParser[29]。

7 结论

在这项工作中, 我们提出了Transformer, 这是第一个完全基于注意力的序列转换模型, 它用多头自注意力机制取代了编码器-解码器架构中最常用的循环层。

对于翻译任务, Transformer的训练速度显著快于基于循环或卷积层的架构。在WMT 2014英德翻译和WMT 2014英法翻译任务上, 我们都达到了新的最先进水平。在前者任务中, 我们的最佳模型甚至超越了所有先前报道的集成模型。

我们对基于注意力机制模型的未来感到兴奋, 并计划将其应用于其他任务。我们计划将Transformer扩展到涉及文本以外的输入和输出模态的问题上, 并研究局部、受限的注意力机制, 以高效处理图像、音频和视频等大型输入和输出。使生成过程减少序列化是我们的另一个研究目标。

我们用于训练和评估模型的代码可在 <https://github.com/tensorflow/tensor2tensor> 获取。

致谢 我们感谢 Nal Kalchbrenner 和 Stephan Gouws 提供的富有成效的意见、修改和启发。

参考文献

- [1] Jimmy Lei Ba, Jamie Ryan Kiros, and Geoffrey E Hinton. 层归一化。 *arXiv preprint arXiv:1607.06450*, 2016.
- [2] Dzmitry Bahdanau, Kyunghyun Cho, and Yoshua Bengio. 通过联合学习对齐和翻译实现神经机器翻译。 *CoRR*, abs/1409.0473, 2014.
- [3] Denny Britz, Anna Goldie, Minh-Thang Luong, and Quoc V. Le. 大规模探索神经机器翻译架构。 *CoRR*, abs/1703.03906, 2017.
- [4] Jianpeng Cheng, Li Dong, and Mirella Lapata. 用于机器阅读的长短期记忆网络。 *arXiv preprint arXiv:1601.06733*, 2016.

- [5] Kyunghyun Cho, Bart van Merriënboer, Çaglar Gulcehre, Fethi Bougares, Holger Schwenk, and Yoshua Bengio. 使用rnn编码器-解码器学习短语表示用于统计机器翻译。 *CoRR*, abs/1406.1078, 2014.
- [6] Francois Chollet. Xception: 深度学习与深度可分离卷积。 *arXiv preprint arXiv:1610.02357*, 2016.
- [7] Junyoung Chung, Çaglar Gülçehre, Kyunghyun Cho, and Yoshua Bengio. 门控循环神经网络在序列建模上的经验评估。 *CoRR*, abs/1412.3555, 2014.
- [8] Chris Dyer, Adhiguna Kuncoro, Miguel Ballesteros, and Noah A. Smith. 循环神经网络语法。载于 *Proc. of NAACL*, 2016.
- [9] Jonas Gehring, Michael Auli, David Grangier, Denis Yarats, and Yann N. Dauphin. 卷积序列到序列学习。 *arXiv preprint arXiv:1705.03122v2*, 2017.
- [10] Alex Graves. 使用循环神经网络生成序列。 *arXiv 预印本 arXiv:1308.0850*, 2013.
- [11] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. 用于图像识别的深度残差学习。收录于《IEEE计算机视觉与模式识别会议论文集》，第770–778页，2016年。
- [12] Sepp Hochreiter, Yoshua Bengio, Paolo Frasconi, and Jürgen Schmidhuber. 循环网络中的梯度流：学习长期依赖的困难，2001.
- [13] Sepp Hochreiter 和 Jürgen Schmidhuber. 长短期记忆。 *Neural computation*, 9(8):1735–1780, 1997.
- [14] Zhongqiang Huang 和 Mary Harper. 跨语言的隐式标注自训练PCFG语法。收录于《2009年自然语言处理经验方法会议论文集》，第832–841页。ACL, 2009年8月。
- [15] Rafal Jozefowicz, Oriol Vinyals, Mike Schuster, Noam Shazeer, 和 Yonghui Wu. 探索语言模型的极限。 *arXiv 预印本 arXiv:1602.02410*, 2016.
- [16] ukasz Kaiser 和 Samy Bengio. 主动记忆能否取代注意力？载于《神经信息处理系统进展》（NIPS），2016年。
- [17] ukasz Kaiser 和 Ilya Sutskever. 神经GPU学习算法。收录于《国际学习表征会议 (ICLR)》，2016.
- [18] Nal Kalchbrenner, Lasse Espeholt, Karen Simonyan, Aaron van den Oord, Alex Graves, and Koray Kavukcuoglu. 线性时间内的神经机器翻译。 *arXiv preprint arXiv:1610.10099v2*, 2017.
- [19] Yoon Kim, Carl Denton, Luong Hoang, and Alexander M. Rush. 结构化注意力网络。载于《国际学习表征会议》，2017.
- [20] Diederik Kingma 和 Jimmy Ba. Adam: 一种随机优化方法。载于 *ICLR*, 2015.
- [21] Oleksii Kuchaiev 和 Boris Ginsburg. LSTM网络的分解技巧。 *arXiv preprint arXiv:1703.10722*, 2017.
- [22] Zhouhan Lin, Minwei Feng, Cicero Nogueira dos Santos, Mo Yu, Bing Xiang, Bowen Zhou, 和 Yoshua Bengio. 一种结构化自注意力句子嵌入。 *arXiv 预印本 arXiv:1703.03130*, 2017.
- [23] Minh-Thang Luong, Quoc V. Le, Ilya Sutskever, Oriol Vinyals, and Lukasz Kaiser. 多任务序列到序列学习。 *arXiv preprint arXiv:1511.06114* 2015.
- [24] Minh-Thang Luong, Hieu Pham, and Christopher D Manning. 基于注意力的神经机器翻译的有效方法。 *arXiv preprint arXiv:1508.04025*, 2015.

- [25] Mitchell P Marcus, Mary Ann Marcinkiewicz, and Beatrice Santorini. Building a large annotated corpus of english: The penn treebank. *Computational linguistics*, 19(2):313–330, 1993.
- [26] David McClosky, Eugene Charniak, 和 Mark Johnson. 用于解析的有效自训练. 收录于 *NAACL人类语言技术会议主会论文集*, 第152–159页. ACL, 2006年6月.
- [27] Ankur Parikh, Oscar Täckström, Dipanjan Das, and Jakob Uszkoreit. A decomposable attention model. In *Empirical Methods in Natural Language Processing*, 2016.
- [28] Romain Paulus, Caiming Xiong, and Richard Socher. A deep reinforced model for abstractive summarization. *arXiv preprint arXiv:1705.04304*, 2017.
- [29] Slav Petrov, Leon Barrett, Romain Thibaux, and Dan Klein. 学习准确、紧凑且可解释的树标注. 载于《第21届国际计算语言学会议暨第44届ACL年会论文集》, 第433–440页. ACL, 2006年7月.
- [30] Ofir Press 和 Lior Wolf. 使用输出嵌入来改进语言模型. *arXiv 预印本 arXiv:1608.05859*, 2016.
- [31] Rico Sennrich, Barry Haddow, and Alexandra Birch. 使用子词单元的稀有词神经网络机器翻译. *arXiv preprint arXiv:1508.07909*, 2015.
- [32] Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziarczyk, Andy Davis, Quoc Le, Geoffrey Hinton, and Jeff Dean. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. *arXiv preprint arXiv:1701.06538*, 2017.
- [33] Nitish Srivastava, Geoffrey E Hinton, Alex Krizhevsky, Ilya Sutskever, and Ruslan Salakhutdinov. Dropout: a simple way to prevent neural networks from overfitting. *Journal of Machine Learning Research*, 15(1):1929–1958, 2014.
- [34] Sainbayar Sukhbaatar, Arthur Szlam, Jason Weston, 和 Rob Fergus. 端到端记忆网络. 载于 C. Cortes, N. D. Lawrence, D. D. Lee, M. Sugiyama, 和 R. Garnett, 编, *Advances in Neural Information Processing Systems 28*, 第 2440–2448 页. Curran Associates, Inc., 2015.
- [35] Ilya Sutskever, Oriol Vinyals, and Quoc VV Le. 神经网络的序列到序列学习. 载于《神经信息处理系统进展》, 第3104–3112页, 2014年.
- [36] Christian Szegedy, Vincent Vanhoucke, Sergey Ioffe, Jonathon Shlens, and Zbigniew Wojna. 重新思考计算机视觉的inception架构. *CoRR*, abs/1512.00567, 2015.
- [37] Vinyals & Kaiser, Koo, Petrov, Sutskever, and Hinton. 语法作为一种外语. 载于《神经信息处理系统进展》, 2015.
- [38] Yonghui Wu, Mike Schuster, Zhifeng Chen, Quoc V Le, Mohammad Norouzi, Wolfgang Macherey, Maxim Krikun, Yuan Cao, Qin Gao, Klaus Macherey, 等. Google的神经机器翻译系统: 弥合人类与机器翻译之间的差距. *arXiv preprint arXiv:1609.08144*, 2016.
- [39] 周杰, 曹颖, 王旭光, 李鹏, 和徐伟. 具有前馈连接的深度循环模型用于神经机器翻译. *CoRR*, abs/1606.04199, 2016.
- [40] Muhua Zhu, Yue Zhang, Wenliang Chen, Min Zhang, and Jingbo Zhu. Fast and accurate shift-reduce constituent parsing. In *Proceedings of the 51st Annual Meeting of the ACL (Volume 1: Long Papers)*, pages 434–443. ACL, August 2013.

注意力可视化

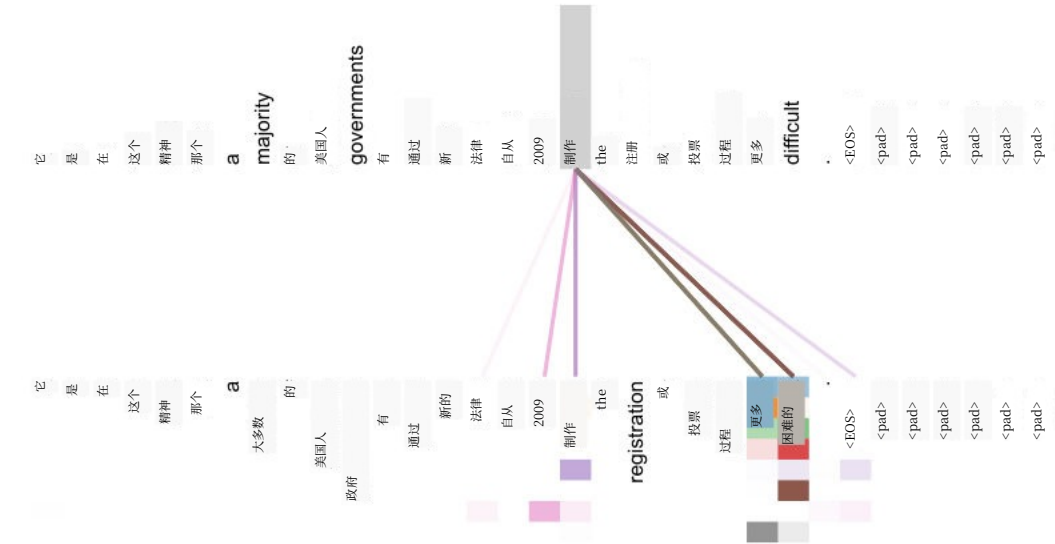


图3：在6层中的第5层编码器自注意力机制中，遵循长距离依赖关系的注意力机制示例。许多注意力头关注动词“making”的远距离依赖关系，补全短语“making...more difficult”。此处仅显示单词“making”的注意力。不同颜色代表不同的注意力头。建议彩色查看。

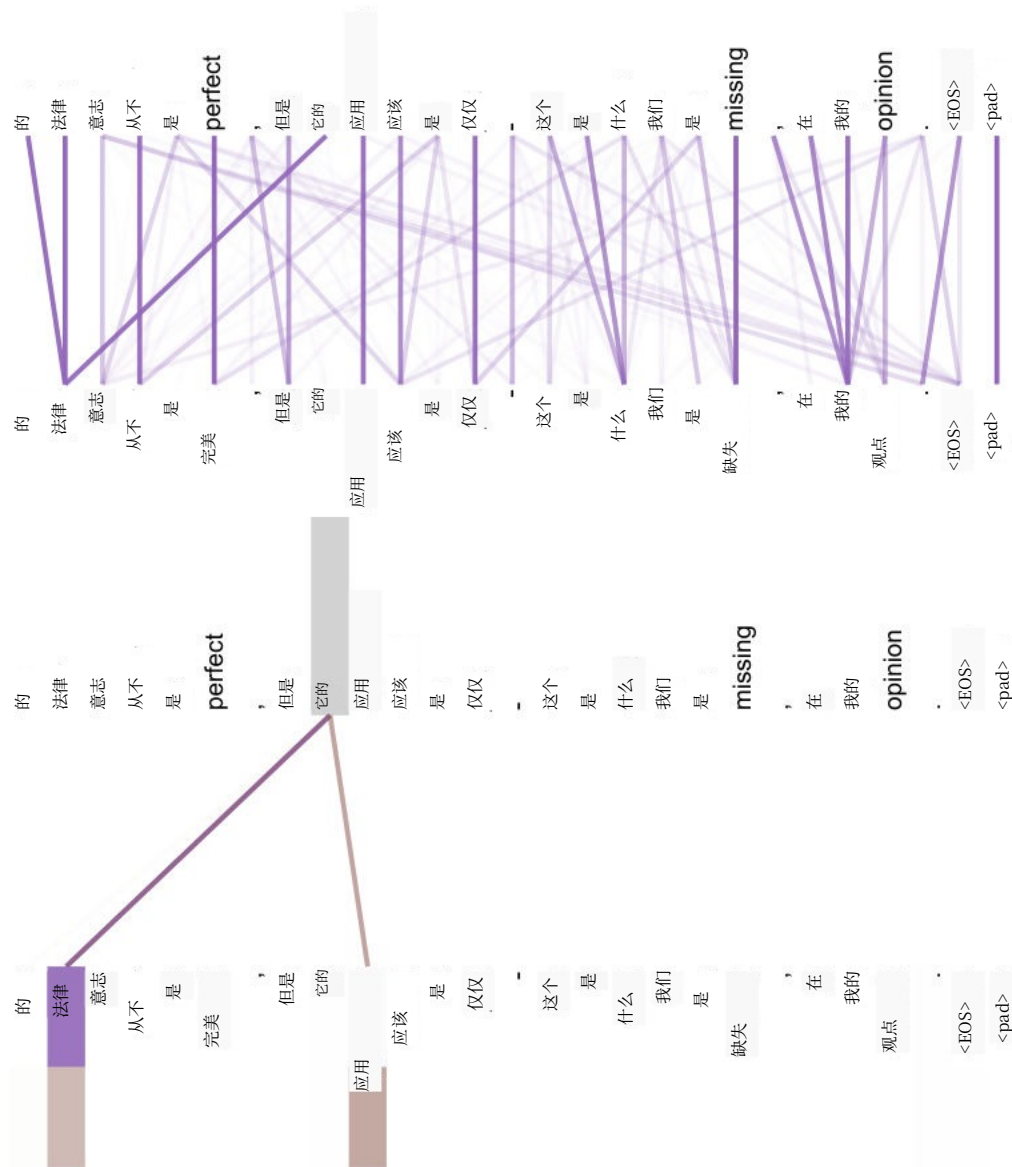


图4：两个注意力头，同样位于6层中的第5层，显然参与了指代消解。顶部：第5个头的完整注意力。底部：仅针对单词“its”的第5和第6个注意力头的孤立注意力。注意，这些注意力在该单词上非常尖锐。

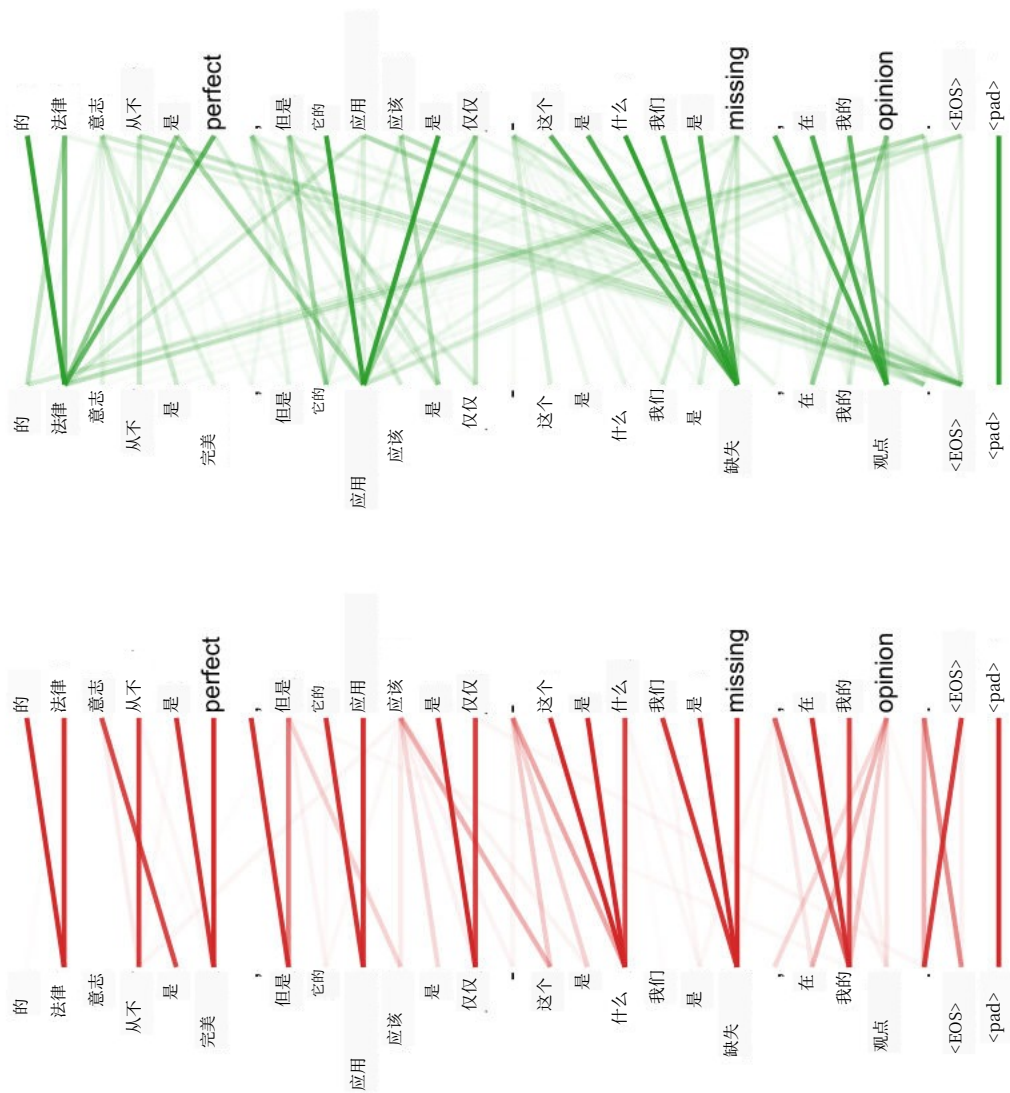


图5: 许多注意力头表现出似乎与句子结构相关的行为。我们在上方给出了两个这样的例子, 它们来自6层编码器自注意力机制中两个不同的头。这些头显然学会了执行不同的任务。